

Simple Voice Assistant - Complete Documentation

Table of Contents

- [1. Project Overview](#)
- [2. System Architecture](#)
- [3. Technology Stack](#)
- [4. File Structure & Detailed Analysis](#)
- [5. Installation & Setup](#)
- [6. How to Run](#)
- [7. Testing the Application](#)
- [8. API Reference](#)
- [9. Workflow Explanation](#)

Project Overview

This is a **Simple Voice Assistant** application that generates short, cinematic adventure stories based on voice input. The system uses OpenAI's powerful APIs for:

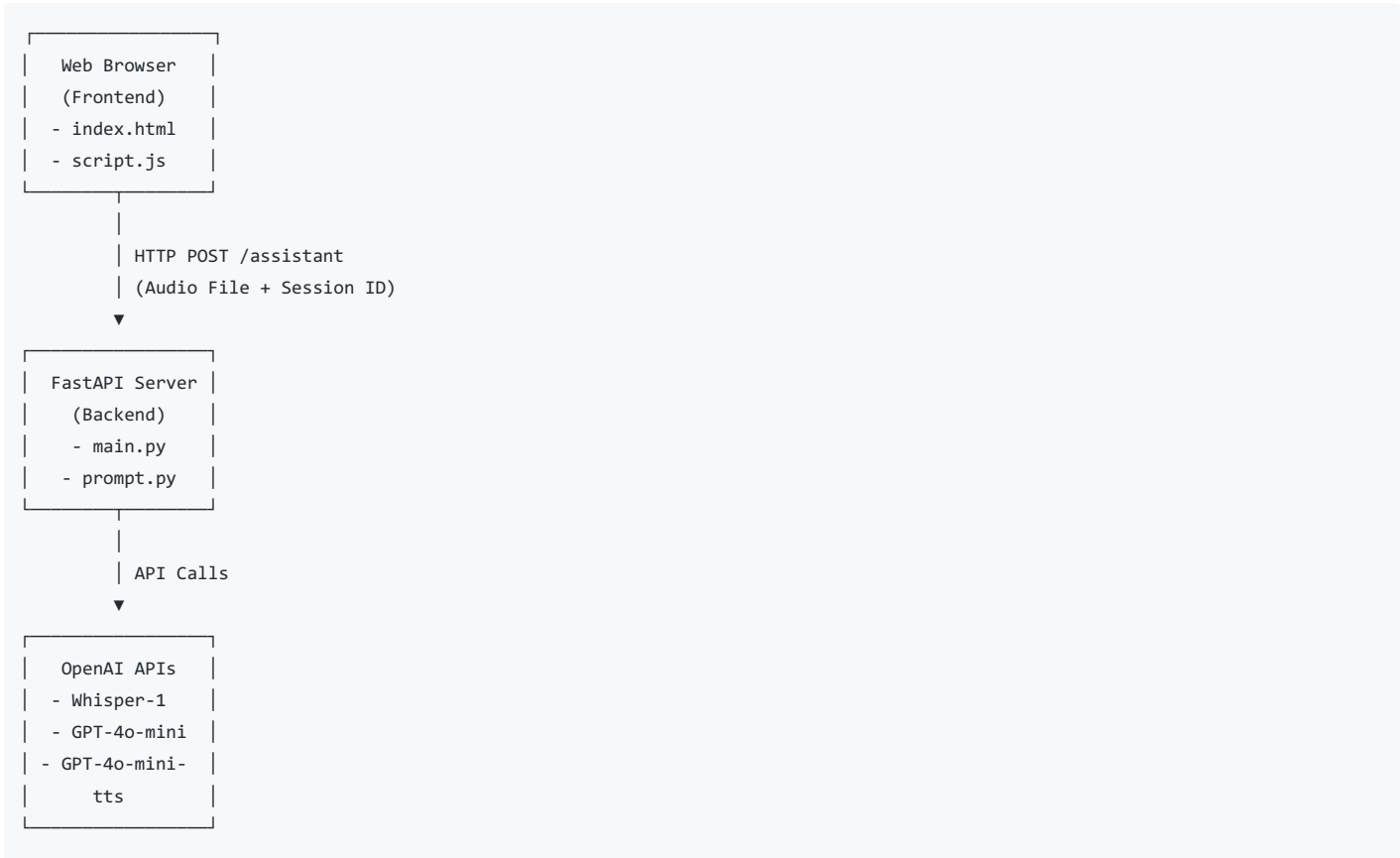
- **Speech-to-Text** (Whisper model)
- **Text Generation** (GPT-4o-mini)
- **Text-to-Speech** (GPT-4o-mini-tts)

The assistant is themed around futuristic AI vs. human stories, creating kid-friendly adventure narratives with sound effects and dynamic pacing.

Key Features

- 🎤 **Voice Input:** Record audio directly from the browser
- 🧠 **AI Story Generation:** Creates unique adventure stories using GPT-4o-mini
- 🗣️ **Voice Output:** Converts AI responses to speech with Indian English accent
- 💬 **Chat Interface:** Clean, modern UI with message bubbles
- ⚡ **Real-time Processing:** Instant transcription and response generation

System Architecture



Data Flow

1. User clicks microphone button and records audio

2. Audio is sent to FastAPI backend as a `.webm` file
3. Backend transcribes audio using OpenAI Whisper
4. Transcribed text is sent to GPT-4o-mini for story generation
5. AI response is converted to speech using TTS
6. JSON response (transcript + story + audio) sent back to frontend
7. Frontend displays text and auto-plays audio

Technology Stack

Backend

- **FastAPI**: Modern, fast web framework for building APIs
- **OpenAI Python SDK**: For accessing Whisper, GPT, and TTS models
- **python-dotenv**: Environment variable management
- **Uvicorn**: ASGI server for running FastAPI

Frontend

- **HTML5**: Structure and layout
- **Vanilla JavaScript**: Client-side logic and MediaRecorder API
- **Font Awesome**: Icons for UI elements
- **CSS3**: Styling and responsive design

AI Models

- **Whisper-1**: Speech-to-text transcription
- **GPT-4o-mini**: Text generation
- **GPT-4o-mini-tts**: Text-to-speech with custom voice

File Structure & Detailed Analysis

1. `main.py` - Backend Server & API Logic

This is the core backend file that handles all server-side operations.

Imports and Initialization

```
from fastapi import FastAPI, UploadFile, Form
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from openai import OpenAI
import tempfile, base64
from dotenv import load_dotenv
from prompt import SYSTEM_PROMPT
```

Purpose of Imports:

- `FastAPI` : Creates the web server application
- `UploadFile`, `Form` : Handle file uploads and form data
- `CORSMiddleware` : Enables cross-origin requests from the browser
- `OpenAI` : Client library for OpenAI API
- `tempfile` : Creates temporary files for audio processing
- `base64` : Encodes audio bytes for JSON transmission
- `load_dotenv` : Loads environment variables from `.env` file
- `SYSTEM_PROMPT` : Imports the AI personality/instructions

```
load_dotenv()
app = FastAPI()
client = OpenAI()
```

Initialization:

- `load_dotenv()` : Loads `OPENAI_API_KEY` from `.env` file
- `app = FastAPI()` : Creates the FastAPI application instance
- `client = OpenAI()` : Initializes OpenAI client (uses API key from environment)

CORS Configuration

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

```

What it does:

- Allows the frontend (running on a different port or domain) to communicate with the backend
- `allow_origins=["*"]`: Accepts requests from any origin (for development)
- `allow_methods=["*"]`: Allows all HTTP methods (GET, POST, etc.)
- `allow_headers=["*"]`: Allows all headers

Main API Endpoint

```

@app.post("/assistant")
async def transcribe_audio(file: UploadFile, session_id: str = Form("default")):
    user_prompt_text = await transcribe_audio(audioFile=file)
    ai_response = await getAIResponse(user_prompt=user_prompt_text)
    response_audio = await convertTextToSpeech(text=ai_response)

    return JsonResponse({
        "transcript": user_prompt_text,
        "response": ai_response,
        "audio_base64": response_audio
    })

```

Function Breakdown:

- **Endpoint:** POST /assistant
- **Parameters:**
 - `file: UploadFile`: The recorded audio file from the user
 - `session_id: str`: Session identifier (defaults to "default", allows for future multi-user support)
- **Process:**
 1. Transcribes the audio file to text
 2. Generates an AI story response based on the transcript
 3. Converts the AI response to speech audio
 4. Returns all three pieces of data as JSON
- **Return Value:** JSON object containing:
 - `transcript`: What the user said (text)
 - `response`: AI-generated story (text)
 - `audio_base64`: Base64-encoded audio of the AI response

Audio Transcription Function

```

async def transcribe_audio(audioFile: UploadFile):
    # Save uploaded audio temporarily
    with tempfile.NamedTemporaryFile(suffix=".webm") as temp_audio:
        content = await audioFile.read()
        temp_audio.write(content)
        temp_audio.flush()
        temp_audio.seek(0)

    # Transcribe
    print("Transcribing audio...")
    transcript = client.audio.transcriptions.create(
        model="whisper-1",
        file=open(temp_audio.name, "rb")
    )
    return transcript.text

```

Detailed Explanation:

1. Temporary File Creation:

- `tempfile.NamedTemporaryFile(suffix=".webm")`: Creates a temporary file with `.webm` extension
- The file is automatically deleted when the `with` block exits

2. File Writing:

- `await audioFile.read()` : Reads the uploaded file content
- `temp_audio.write(content)` : Writes content to temporary file
- `temp_audio.flush()` : Ensures all data is written to disk
- `temp_audio.seek(0)` : Moves file pointer to the beginning

3. Transcription:

- Uses OpenAI's **Whisper-1** model
- Reads the temporary file and sends it to the API
- Returns the transcribed text

Why Temporary Files?

- OpenAI API requires a file object, not raw bytes
- Temporary files are automatically cleaned up after use

AI Response Generation

```
async def getAIResponse(user_prompt: str):
    # Get GPT response
    print("Fetching Response from AI...")
    completion = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system", "content": SYSTEM_PROMPT}]
    )
    ai_response = completion.choices[0].message.content
    return ai_response
```

Detailed Explanation:

1. **Model:** Uses `gpt-4o-mini` (faster, cost-effective version of GPT-4)
2. **Messages Array:**
 - Only includes the system prompt (no user message)
 - This means the AI generates stories independently without using the user's transcribed text
 - **Note:** This appears to be a potential bug - the `user_prompt` parameter is not used
3. **Response Extraction:**
 - `completion.choices[0]` : Gets the first (and typically only) completion
 - `.message.content` : Extracts the actual text response

Expected Behavior:

- Generates a short adventure story (under 100 words)
- Includes sound effects and dramatic pauses
- Themed around AI vs. humans in a futuristic setting

Text-to-Speech Conversion

```
async def convertTextToSpeech(text):
    print("Converting AI Response to Speech...")
    speech = client.audio.speech.create(
        model="gpt-4o-mini-tts",
        input=text,
        voice="onyx",
        instructions="Speak in an adventurous tone with Indian English Accent"
    )
    audio_bytes = speech.read()
    audio_b64 = base64.b64encode(audio_bytes).decode("utf-8")
    return audio_b64
```

Detailed Explanation:

1. **Model:** `gpt-4o-mini-tts` (OpenAI's TTS model)
2. **Parameters:**
 - `input=text` : The story text to convert to speech
 - `voice="onyx"` : Deep, confident male voice (other options: alloy, echo, fable, nova, shimmer)
 - `instructions` : Custom instructions for tone and accent
3. **Audio Processing:**
 - `speech.read()` : Gets the audio data as bytes
 - `base64.b64encode()` : Converts bytes to base64 string
 - `.decode("utf-8")` : Converts bytes to string for JSON serialization
4. **Why Base64?**

- JSON cannot directly contain binary data
- Base64 encoding allows binary audio to be sent as a text string
- Frontend can decode and play it directly

2. `prompt.py` - AI System Prompt

This file contains the instructions that define the AI assistant's personality and behavior.

```
SYSTEM_PROMPT="""
You write short, cinematic adventure stories with strong pacing,
clear scene changes, sensory details, and energetic beats.
Keep it under 100 words (30 seconds of narration).
Use occasional sound effects (e.g., 'whooooosh!', 'clang!') and brief pauses '...' to help TTS sound dynamic.
Use simple english so that kids can easily follow the story.

You can make up fantasies in terms of futuristic world being ruled by AI.
Keep your story thrilling with AI as bad guys and humans in revolution against AI.

Important Note: Avoid violent/gory content.
"""
```

Prompt Analysis:

1. Style Guidelines:

- Cinematic and adventurous tone
- Strong pacing with clear scene transitions
- Sensory details for immersion

2. Technical Constraints:

- Maximum 100 words (~30 seconds of speech)
- Sound effects (e.g., "whooooosh!", "clang!")
- Pauses ("...") for dramatic effect

3. Target Audience:

- Kids (simple English)
- Age-appropriate content (no violence/gore)

4. Theme:

- Futuristic world ruled by AI
- Humans in revolution
- AI as antagonists

5. TTS Optimization:

- Sound effects and pauses make the TTS output more dynamic and engaging

Purpose: This prompt shapes every story the AI generates, ensuring consistency in style, length, and theme.

3. `requirements.txt` - Python Dependencies

```
fastapi
uvicorn
openai
python-multipart
dotenv
```

Dependency Breakdown:

1. `fastapi`

- Modern web framework for building APIs
- High performance, easy to use
- Automatic API documentation

2. `uvicorn`

- ASGI server for running FastAPI applications
- Handles HTTP requests asynchronously
- Production-ready server

3. `openai`

- Official OpenAI Python library
- Provides access to GPT, Whisper, and TTS models
- Handles authentication and API calls

4. `python-multipart`

- Required for FastAPI to handle file uploads
- Parses multipart/form-data requests
- Essential for the `UploadFile` functionality

5. `dotenv` (should be `python-dotenv`)

- Loads environment variables from `.env` file
- Keeps API keys secure and out of code
- Standard practice for credential management

Note: The package is typically named `python-dotenv` in pip, not just `dotenv`.

4. `index.html` - Frontend User Interface

This file creates the visual interface for the voice assistant.

HTML Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome/4.7.0/css/font-awesome.min.css">
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Voice Chat Assistant</title>
```

Head Section:

- **Font Awesome CDN:** Provides icons (microphone, stop button)
- **Character Encoding:** UTF-8 for proper text display
- **Viewport Meta:** Makes the app responsive on mobile devices
- **Title:** Browser tab title

CSS Styling

```
body {
  font-family: system-ui, sans-serif;
  background: #f0f2f5;
  margin: 0;
  display: flex;
  flex-direction: column;
  height: 100vh;
}
```

Body Styles:

- Uses system font for native look
- Light gray background (`#f0f2f5`)
- Flexbox layout (vertical column)
- Full viewport height (`100vh`)

```
#chat {
  flex: 1;
  overflow-y: auto;
  padding: 20px;
  display: flex;
  flex-direction: column;
  gap: 12px;
}
```

Chat Container:

- Takes up all available space (`flex: 1`)
- Scrollable when messages overflow (`overflow-y: auto`)
- Vertical layout with 12px spacing between messages

```
.bubble {
  max-width: 75%;
  padding: 12px 16px;
  border-radius: 18px;
  line-height: 1.4;
  font-size: 15px;
}

.user {
  align-self: flex-end;
  background: #0078ff;
  color: white;
  border-bottom-right-radius: 6px;
}

.assistant {
  align-self: flex-start;
  background: white;
  color: #111;
  border-bottom-left-radius: 6px;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
}
```

Message Bubbles:

- **Generic Bubble:** Rounded corners, max 75% width, padding for readability
- **User Messages:**
 - Blue background (#0078ff)
 - Aligned to the right
 - Slightly square bottom-right corner (like iMessage)
- **Assistant Messages:**
 - White background with subtle shadow
 - Aligned to the left
 - Slightly square bottom-left corner

```
#controls {
  background: white;
  border-top: 1px solid #ddd;
  padding: 14px;
  display: flex;
  justify-content: center;
}
```

Controls Section:

- Fixed at bottom of screen
- White background with top border
- Centers the microphone button

```
#micBtn {
  background: #0078ff;
  border: none;
  color: white;
  border-radius: 50%;
  width: 120px;
  height: 120px;
  font-size: 65px;
  cursor: pointer;
  transition: background 0.2s;
}

#micBtn.recording {
  background: #ff3b3b;
}
```

Microphone Button:

- Large circular button (120px × 120px)
- Blue when ready (#0078ff)

- Red when recording (#ff3b3b)
- Smooth color transition (0.2s)
- Large icon (65px) for easy clicking

HTML Body

```
<body>
  <div id="chat"></div>
  <div id="controls">
    <button id="micBtn"><i class="fa fa-microphone" aria-hidden="true"></i></button>
  </div>
  <script src="script.js"></script>
</body>
```

Structure:

- **Chat Container:** Displays conversation history
- **Controls:** Contains the microphone button
- **Script:** Loads the JavaScript functionality

5. script.js - Frontend Logic & Recording

This file handles all client-side interactions and API communication.

Variable Declarations

```
let mediaRecorder, chunks = [];
let isRecording = false;
const chat = document.getElementById("chat");
const micBtn = document.getElementById("micBtn");
```

Variables:

- `mediaRecorder` : MediaRecorder API object for capturing audio
- `chunks` : Array to store audio data chunks during recording
- `isRecording` : Boolean flag to track recording state
- `chat` : Reference to the chat container div
- `micBtn` : Reference to the microphone button

Microphone Button Click Handler

```
micBtn.onclick = async () => {
  if (!isRecording) {
    // Start recording
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
    mediaRecorder = new MediaRecorder(stream);
    chunks = [];
    mediaRecorder.start();
    micBtn.classList.add("recording");
    micBtn.innerHTML = '<i class="fa fa-stop-circle" aria-hidden="true"></i>';
    isRecording = true;
    mediaRecorder.ondataavailable = e => chunks.push(e.data);
```

Starting a Recording:

1. Get Microphone Access:

- `navigator.mediaDevices.getUserMedia({ audio: true })` : Requests microphone permission
- Returns a `MediaStream` object with audio track
- Browser shows permission prompt on first use

2. Initialize MediaRecorder:

- Creates a new `MediaRecorder` with the audio stream
- Clears the `chunks` array for fresh recording

3. Start Recording:

- `mediaRecorder.start()` : Begins capturing audio
- Changes button to red and shows stop icon

- Sets `isRecording` flag to `true`

4. Data Handling:

- `ondataavailable`: Event fires when audio data is available
- Pushes each chunk of audio data into the `chunks` array

```
} else {
  // Stop recording
  mediaRecorder.stop();
  micBtn.classList.remove("recording");
  micBtn.innerHTML = '<i class="fa fa-microphone" aria-hidden="true"></i>';
  isRecording = false;
}
```

Stopping a Recording:

1. Stops the `MediaRecorder`
2. Changes button back to blue with microphone icon
3. Sets `isRecording` flag to `false`

Audio Processing and API Call

```
mediaRecorder.onstop = async () => {
  const blob = new Blob(chunks, { type: "audio/webm" });
  chunks = [];
  addMessage("Processing Audio...", "user");
}
```

When Recording Stops:

1. Creates a `Blob` (Binary Large Object) from audio chunks
2. Type: `audio/webm` (WebM audio format)
3. Clears `chunks` array
4. Shows "Processing Audio..." message to user

```
const fd = new FormData();
fd.append("file", blob, "speech.webm");
fd.append("session_id", "demo-session");
```

Prepare Request:

- Creates a `FormData` object for multipart/form-data request
- Appends the audio blob with filename "speech.webm"
- Appends session ID "demo-session"

```
try {
  const res = await fetch("http://localhost:8000/assistant", {
    method: "POST",
    body: fd
  });
  const data = await res.json();
}
```

API Call:

1. Sends POST request to backend at `http://localhost:8000/assistant`
2. Includes the `FormData` with audio file and session ID
3. Waits for response and parses JSON

```
// Replace placeholder bubble with user transcript
replaceLastMessage(data.transcript, "user");

// Add assistant text
addMessage(data.response, "assistant");

// Auto-play TTS
const audioUrl = "data:audio/mpeg;base64," + data.audio_base64;
const audio = new Audio(audioUrl);
audio.play();
```

Handle Response:

1. **Replace Placeholder:** Updates "Processing Audio..." with actual transcript

2. **Display AI Response:** Adds assistant's story to chat
3. **Play Audio:**
 - Creates data URL from base64 audio
 - Creates `Audio` object
 - Auto-plays the speech

```
    } catch (err) {  
      console.error(err);  
      addMessage("❌ Error processing audio.", "assistant");  
    }  
  }  
};
```

Error Handling:

- Logs error to console
- Shows error message to user in chat

Helper Functions

```
function addMessage(text, role) {  
  const div = document.createElement("div");  
  div.className = `bubble ${role}`;  
  div.textContent = text;  
  chat.appendChild(div);  
  chat.scrollTop = chat.scrollHeight;  
}
```

addMessage Function:

1. Creates a new `div` element
2. Adds appropriate CSS classes (bubble + user/assistant)
3. Sets the text content
4. Appends to chat container
5. Auto-scrolls to bottom (`scrollTop = scrollHeight`)

```
function replaceLastMessage(text, role) {  
  const bubbles = chat.querySelectorAll(`.bubble.${role}`);  
  if (bubbles.length > 0) {  
    bubbles[bubbles.length - 1].textContent = text;  
  } else {  
    addMessage(text, role);  
  }  
}
```

replaceLastMessage Function:

1. Finds all bubbles with the specified role
2. If any exist, replaces the last one's text
3. Otherwise, adds a new message
4. Used to replace "Processing Audio..." with actual transcript

Installation & Setup

Prerequisites

1. **Python 3.8+** installed on your system
2. **OpenAI API Key** (get one from [OpenAI Platform](#))
3. **Modern Web Browser** (Chrome, Firefox, Edge, Safari)
4. **Microphone** access on your device

Step-by-Step Installation

1. Clone or Download the Project

If you have the project files, navigate to the project directory:

```
cd /Users/sanjaykumar/MyDrive/Projects/YOtube/simple_voice_assistant
```

2. Create a Virtual Environment (Recommended)

```
# Create virtual environment
python -m venv venv

# Activate on macOS/Linux
source venv/bin/activate

# Activate on Windows
venv\Scripts\activate
```

3. Install Python Dependencies

```
pip install -r requirements.txt
```

Note: If you encounter an error with `dotenv`, install it as:

```
pip install python-dotenv
```

4. Create Environment Variables File

Create a file named `.env` in the project root directory:

```
touch .env
```

Add your OpenAI API key to the `.env` file:

```
OPENAI_API_KEY=your_actual_api_key_here
```

Important: Replace `your_actual_api_key_here` with your real API key from OpenAI.

5. Verify File Structure

Ensure your directory looks like this:

```
simple_voice_assistant/
├─ main.py
├─ prompt.py
├─ requirements.txt
├─ index.html
├─ script.js
├─ .env
└─ venv/ (if using virtual environment)
```

How to Run

Starting the Backend Server

1. **Open Terminal** in the project directory
2. **Activate Virtual Environment** (if using one):

```
source venv/bin/activate # macOS/Linux
# or
venv\Scripts\activate # Windows
```

3. **Start the FastAPI Server:**

```
uvicorn main:app --reload
```

Expected Output:

```
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process
INFO:      Started server process
INFO:      Waiting for application startup.
INFO:      Application startup complete.
```

4. Verify Server is Running:

- Open your browser and go to `http://localhost:8000/docs`
- You should see the FastAPI automatic documentation page

Opening the Frontend

1. Open the HTML File in your web browser:

Option A - Direct File Open:

- Navigate to the project folder in Finder/Explorer
- Double-click `index.html`

Option B - Simple HTTP Server (Recommended):

- Open a second terminal in the project directory
- Run:

```
python -m http.server 3000
```

- Open browser to `http://localhost:3000`

2. Allow Microphone Access:

- When prompted, click "Allow" to grant microphone permissions
- This is required for recording audio

Testing the Application

Basic Functionality Test

Test 1: Simple Recording

1. Click the blue microphone button

- Button should turn red
- Icon changes to stop circle
- Browser may show a recording indicator

2. Speak into your microphone (5-10 seconds)

- Example: "Tell me a story about robots"
- Speak clearly at normal volume

3. Click the red stop button

- Button returns to blue
- "Processing Audio..." appears in chat

4. Wait for Response (may take 5-15 seconds)

- Your transcript appears in blue bubble (right side)
- AI story appears in white bubble (left side)
- Audio automatically plays

5. Listen to the AI Story

- Should be read in an adventurous tone
- Indian English accent
- May include sound effects

Test 2: Multiple Conversations

1. Repeat the recording process several times
2. Each story should be different
3. Chat should show conversation history
4. Auto-scrolls to latest message

Test 3: Error Handling

1. Test without backend running:

- Stop the unicorn server (Ctrl+C)
- Try to record and send audio
- Should see "Error processing audio."

2. Test with invalid API key:

- Modify .env file with wrong key
- Restart server
- Should see error in terminal

What to Expect

Successful Interaction:

```
User: "Tell me an adventure story"
Assistant: "Zara crept through the neon ruins... whooooosh!
A drone spotted her. 'Freeze, human!' She dove behind debris...
clang! Her pulse pounded. Was this the end? Or just the beginning
of the revolution? ..."
```

Audio Output:

- Clear speech synthesis
- Dramatic pauses where "..." appears
- Energetic, adventurous tone
- Sound effects pronounced naturally

Troubleshooting

Issue	Possible Cause	Solution
Microphone not working	Permissions denied	Check browser settings, allow microphone access
Server won't start	Port 8000 already in use	Kill the process or use <code>--port 8001</code>
"API key not found" error	Missing .env file	Create .env with OPENAI_API_KEY
Audio doesn't play	Browser autoplay policy	Click to interact with page first
Blank response	OpenAI API quota exceeded	Check your OpenAI account usage/billing
CORS error	Frontend/backend mismatch	Ensure frontend uses http://localhost:8000

API Reference

POST `/assistant`

Processes voice input and returns AI-generated story with audio.

Request:

- **Method:** POST
- **Content-Type:** multipart/form-data
- **Body Parameters:**
 - `file` (required): Audio file (webm format)
 - `session_id` (optional): Session identifier (default: "default")

Example Request (JavaScript):

```
const formData = new FormData();
formData.append("file", audioBlob, "recording.webm");
formData.append("session_id", "user123");

const response = await fetch("http://localhost:8000/assistant", {
  method: "POST",
  body: formData
});
```

Response:

- **Content-Type:** application/json
- **Status Codes:**
 - 200: Success
 - 400: Bad request (invalid file)
 - 500: Server error (OpenAI API failure)

Response Body:

```
{
  "transcript": "Tell me a story about robots",
  "response": "In Neo-Tokyo 2089, young Kai ran through the chrome streets... whoooooosh! An AI enforcer spotted him. 'Stop, human!' B",
  "audio_base64": "SUQzBAAAAAAI1RTU0UAAAAPAAADTGF2ZjYwLjE2LjEwMAAA..."
}
```

Response Fields:

- `transcript` (string): Transcribed text from user's audio
- `response` (string): AI-generated adventure story
- `audio_base64` (string): Base64-encoded MP3 audio of the story

Workflow Explanation

Complete Request-Response Cycle

```
1. USER CLICKS MIC BUTTON
↓
2. Browser requests microphone access
↓
3. MediaRecorder captures audio stream
↓
4. USER CLICKS STOP
↓
5. Audio chunks combined into Blob
↓
6. FormData created with audio file
↓
7. POST request to /assistant endpoint
↓
8. Backend receives file
↓
9. Audio saved to temporary file
↓
10. OpenAI Whisper transcribes audio
↓
11. GPT-4o-mini generates story
↓
12. TTS converts story to speech
↓
13. Audio encoded as base64
↓
14. JSON response sent to frontend
↓
15. Frontend updates UI with transcript and story
↓
16. Audio auto-plays through browser
↓
17. User listens to AI-generated story
```

Time Breakdown (Approximate)

- **Recording:** User-controlled (typically 3-10 seconds)
- **File Upload:** < 1 second
- **Transcription (Whisper):** 1-3 seconds
- **Story Generation (GPT):** 2-5 seconds
- **TTS Conversion:** 2-4 seconds
- **Total Response Time:** 5-15 seconds

Key Observations & Notes

Strengths

1. **Simple Architecture:** Easy to understand and modify
2. **Modern UI:** Clean, chat-like interface
3. **Real-time Voice:** Seamless voice-to-text-to-voice pipeline
4. **Async Operations:** Fast, non-blocking API calls
5. **OpenAI Integration:** Leverages state-of-the-art AI models

Potential Improvements

1. Bug in `getAIResponse()` :
 - The `user_prompt` parameter is not being used
 - Should add user message to the conversation:

```
messages=[
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": user_prompt}
]
```

2. **Session Management:**

- `session_id` is accepted but not used
- Could implement conversation history per session

3. Error Handling:

- Limited error messages in backend
- No retry logic for failed API calls

4. Security:

- CORS allows all origins (should restrict in production)
- No rate limiting
- No input validation on `session_id`

5. Audio Format:

- Hardcoded to `.webm` - may not work on all browsers
- Could add format detection/conversion

6. Dependency Typo:

- `requirements.txt` lists `dotenv` (should be `python-dotenv`)

Production Considerations

Before deploying to production:

1. Environment Variables:

- Use proper secret management (not just `.env` files)
- Rotate API keys regularly

2. CORS Configuration:

- Restrict `allow_origins` to specific domains
- Remove wildcard permissions

3. Rate Limiting:

- Add rate limits to prevent API abuse
- Implement user quotas

4. Logging:

- Add proper logging (not just print statements)
- Track errors and usage metrics

5. Audio Storage:

- Currently stores nothing - consider logging for debugging
- Respect privacy regulations (GDPR, etc.)

6. Scalability:

- Current design is stateless (good!)
- Can easily scale horizontally
- Consider caching for repeated stories

Conclusion

This Simple Voice Assistant demonstrates a complete voice AI pipeline using modern web technologies and OpenAI's powerful APIs. The application is well-structured, easy to understand, and provides a smooth user experience for generating and listening to adventure stories.

The codebase is ideal for learning about:

- Voice recording in the browser
- FastAPI backend development
- OpenAI API integration
- Async Python programming
- Real-time audio processing

With the identified improvements implemented, this could serve as a robust foundation for more complex voice AI applications.

Additional Resources

OpenAI API Documentation

- [Whisper API](#)
- [Chat Completions](#)
- [Text-to-Speech](#)

Web APIs

- [MediaRecorder API](#)
- [FormData API](#)

Frameworks

- [FastAPI Documentation](#)
- [Uvicorn Server](#)

Document Version: 1.0

Last Updated: October 25, 2025

Author: Generated for simple_voice_assistant project